we just perform one removal, although with possibly several rotations.

And that's actually one negative aspect: if we delete a node close to the root, we will need to push it down for several layers till it reaches the root.

The worst-case running time for the remove algorithm is, in other words, `O(h)`, where `h` is the height of the treap.

As a consequence, it becomes particularly important, as you can imagine, that the height of the tree is kept as small as possible.

As you can see from our examples, using the treap for storing both keys and meaningful priorities might tend to produce an unbalanced tree, and removing a node might make the tree even more unbalanced, because of the many rotations, starting from an already bad situation.

3.3.6 Top, Peak, Update

The remaining methods in class `Treap`'s API are easier to implement. Method peak is trivial to implement, it's exactly the same as for regular heaps, the only difference being in how we access the heap's root.

If we also need to offer implement method `top`, to make sure our treap can seamlessly replace a heap, we can leverage method remove and write almost a one-liner, as shown in listing 3.7.

Listing 3.7 method Treap::top

```
function top(treap)
                              #1
  throw-if treap.root == null
                                                    #2
```